

ASP.NET

Unit-4: Advanced Web Forms

Comprehensive Study Notes

BCA Semester V — Himachal Pradesh University

Contents

1. List Controls
2. Validation Controls
3. Data Binding
4. Data Controls
5. Master Pages
6. Navigation
7. Themes & Skins
8. Globalization & Localization
9. Security
10. Questions

1. List Controls

1.1 Theory

List controls in ASP.NET display collections of data items that users can select or view. They are divided into:

1. **Simple List Controls** — display items as a simple list (DropDownList, ListBox, CheckBoxLayout, RadioButtonList, BulletedList)
2. **Template List Controls** — display items using custom templates (Repeater, DataList)

1.2 DropDownList

Displays a drop-down list where users can select one item. Supports `AutoPostBack` and data binding.

```
<asp:DropDownList ID="ddlCountry" runat="server" AutoPostBack="true">
  <asp:ListItem Text="India" Value="IN" />
  <asp:ListItem Text="USA" Value="US" />
  <asp:ListItem Text="UK" Value="GB" />
</asp:DropDownList>
<asp:Label ID="lblSelected" runat="server" />
```

```
' Code-Behind (VB.NET)
Protected Sub ddlCountry_SelectedIndexChanged(ByVal sender As Object, ByVal e
As EventArgs)
    lblSelected.Text = "You selected: " & ddlCountry.SelectedItem.Text
End Sub

' Data Binding
Protected Sub BindDropDown()
    ddlCountry.DataSource = GetCountries()
    ddlCountry.DataTextField = "CountryName"
    ddlCountry.DataValueField = "CountryCode"
    ddlCountry.DataBind()
End Sub
```

1.3 ListBox

Displays items in a box. Supports single or multiple selection via `SelectionMode`. Use `Rows` to set visible rows.

```
<asp:ListBox ID="lstSingle" runat="server" Rows="5" SelectionMode="Single">
    <asp:ListItem Text="C" />
    <asp:ListItem Text="C++" />
    <asp:ListItem Text="Java" />
    <asp:ListItem Text="C#" />
</asp:ListBox>

<asp:ListBox ID="lstMultiple" runat="server" Rows="4"
SelectionMode="Multiple">
    <asp:ListItem Text="Music" />
    <asp:ListItem Text="Sports" />
    <asp:ListItem Text="Reading" />
</asp:ListBox>
<asp:Button ID="btnGetSelection" runat="server" Text="Get Selected" />
<asp:Label ID="lblSelections" runat="server" />
```

```
' VB.NET Code-Behind
Protected Sub btnGetSelection_Click(ByVal sender As Object, ByVal e As
EventArgs)
    Dim sb As New StringBuilder()
    For Each li As ListItem In lstMultiple.Items
        If li.Selected Then sb.Append(li.Text & ", ")
    Next
    lblSelections.Text = "Selected: " & sb.ToString()
End Sub
```

1.4 CheckBoxList

Displays a group of checkboxes. Users can select multiple items. Renders as a `<table>` by default.

```
<asp:CheckBoxList ID="cblInterests" runat="server"
RepeatDirection="Horizontal">
    <asp:ListItem Text="Music" Value="Music" />
    <asp:ListItem Text="Sports" Value="Sports" />
    <asp:ListItem Text="Reading" Value="Reading" />
    <asp:ListItem Text="Travel" Value="Travel" />
</asp:CheckBoxList>
```

```
<asp:Button ID="btnShow" runat="server" Text="Show Selected" />
<asp:Label ID="lblInterests" runat="server" />
```

```
' VB.NET Code-Behind
Protected Sub btnShow_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim sb As New StringBuilder()
    For Each li As ListItem In cblInterests.Items
        If li.Selected Then sb.Append(li.Value & ", ")
    Next
    lblInterests.Text = "Interests: " & sb.ToString()
End Sub
```

1.5 RadioButtonList

Displays radio buttons. Users can select only one option. Useful for mutually exclusive choices.

```
<asp:RadioButtonList ID="rblPayment" runat="server"
RepeatDirection="Vertical">
    <asp:ListItem Text="Credit Card" Value="CC" />
    <asp:ListItem Text="Debit Card" Value="DC" />
    <asp:ListItem Text="Net Banking" Value="NB" />
    <asp:ListItem Text="COD" Value="COD" />
</asp:RadioButtonList>
<asp:Label ID="lblPayment" runat="server" />
```

```
' VB.NET Code-Behind
Protected Sub rblPayment_SelectedIndexChanged(ByVal sender As Object, ByVal e
As EventArgs)
    lblPayment.Text = "Payment method: " & rblPayment.SelectedItem.Text
End Sub
```

1.6 BulletedList

Displays items as a bulleted list (like HTML `<ul>`). Supports `BulletStyle` (Numbers, LowerRoman, UpperAlpha, etc.) and `DisplayMode` (Text, LinkButton, HyperLink).

```
<asp:BulletedList ID="blLinks" runat="server" DisplayMode="HyperLink"
BulletStyle="Disc" Target="_blank">
    <asp:ListItem Text="Google" Value="http://www.google.com" />
    <asp:ListItem Text="Yahoo" Value="http://www.yahoo.com" />
```

```
<asp:ListItem Text="Bing" Value="http://www.bing.com" />
</asp:BulletedList>
```

1.7 List Controls Comparison

Control	Selection	Display	Use Case
DropDownList	Single	Drop-down	Limited space, single selection
ListBox	Single/Multiple	Box list	Multiple items visible
CheckBoxList	Multiple	Checkboxes	Multiple choices
RadioButtonList	Single	Radio buttons	One choice from options
BulletedList	None	Bulleted items	Display list of items

2. Validation Controls

2.1 Theory

Validation controls provide automatic **client-side and server-side validation** without writing JavaScript. When a postback occurs, validation runs on the server; if all validators pass, the page is valid.

2.2 Common Properties

Property	Description
ControlToValidate	ID of the control to validate
ErrorMessage	Error message shown in ValidationSummary
Display	Display behavior: Static, Dynamic, None
Text	Inline error message shown next to the control
SetFocusOnError	If true, sets focus to the invalid control
IsValid	Gets whether validation succeeded
Validate()	Performs validation programmatically
ForeColor	Color of error message (default: Red)

2.3 RequiredFieldValidator

Ensures the user enters a value in the control.

```
<asp:TextBox ID="txtName" runat="server" />
<asp:RequiredFieldValidator ID="rfvName" runat="server"
    ControlToValidate="txtName" ErrorMessage="Name is required."
    Display="Dynamic" ForeColor="Red" SetFocusOnError="true" />
```

```
<!-- For DropDownList -->
<asp:DropDownList ID="ddlCountry" runat="server">
    <asp:ListItem Text="--Select--" Value="" />
    <asp:ListItem Text="India" Value="IN" />
</asp:DropDownList>
<asp:RequiredFieldValidator ID="rfvCountry" runat="server"
    ControlToValidate="ddlCountry" InitialValue=""
    ErrorMessage="Please select a country." />
```

2.4 CompareValidator

Compares the value of one control with another control or a constant value. Uses operator (Equal, NotEqual, GreaterThan, etc.), Type (String, Integer, Double, Date, Currency).

```
<!-- Compare with constant value (Minimum age 18) -->
<asp:TextBox ID="txtAge" runat="server" />
<asp:CompareValidator ID="cvAge" runat="server"
    ControlToValidate="txtAge" ValueToCompare="18"
    Operator="GreaterThanEqual" Type="Integer"
    ErrorMessage="Age must be 18 or above." />

<!-- Compare with another control (Password confirmation) -->
<asp:TextBox ID="txtPassword" runat="server" TextMode="Password" />
<asp:TextBox ID="txtConfirmPassword" runat="server" TextMode="Password" />
<asp:CompareValidator ID="cvPassword" runat="server"
    ControlToValidate="txtConfirmPassword"
    ControlToCompare="txtPassword" Operator="Equal"
    ErrorMessage="Passwords do not match." />

<!-- Data Type check -->
<asp:TextBox ID="txtSalary" runat="server" />
<asp:CompareValidator ID="cvSalary" runat="server"
    ControlToValidate="txtSalary" Operator="DataTypeCheck" Type="Double"
    ErrorMessage="Enter a valid salary amount." />
```

2.5 RangeValidator

Checks if the value falls within a specified range. Uses MinimumValue , MaximumValue , and Type .


```

<!-- Numeric Range (0 to 100) -->
<asp:TextBox ID="txtMarks" runat="server" />
<asp:RangeValidator ID="rvMarks" runat="server"
    ControlToValidate="txtMarks" MinimumValue="0" MaximumValue="100"
    Type="Integer" ErrorMessage="Marks must be between 0 and 100." />

<!-- Date Range -->
<asp:TextBox ID="txtDate" runat="server" />
<asp:RangeValidator ID="rvDate" runat="server"
    ControlToValidate="txtDate" MinimumValue="2020-01-01" MaximumValue="2025-
12-31"
    Type="Date" ErrorMessage="Enter date between 2020 and 2025." />

<!-- Currency Range -->
<asp:TextBox ID="txtPrice" runat="server" />
<asp:RangeValidator ID="rvPrice" runat="server"
    ControlToValidate="txtPrice" MinimumValue="10.00" MaximumValue="5000.00"
    Type="Double" ErrorMessage="Price must be between $10 and $5000." />

```

2.6 RegularExpressionValidator

Validates that the input matches a specific regular expression pattern using `ValidationExpression`.

```

<!-- Email Validation -->
<asp:TextBox ID="txtEmail" runat="server" />
<asp:RegularExpressionValidator ID="revEmail" runat="server"
    ControlToValidate="txtEmail"
    ValidationExpression="\w+([-+.']\w+)*@\w+([-.\w+)*\.\w+([-.\w+)*"
    ErrorMessage="Enter a valid email address." />

<!-- Phone Number Validation (###-###-####) -->
<asp:TextBox ID="txtPhone" runat="server" />
<asp:RegularExpressionValidator ID="revPhone" runat="server"
    ControlToValidate="txtPhone"
    ValidationExpression="^\d{3}-\d{3}-\d{4}$"
    ErrorMessage="Enter phone as XXX-XXX-XXXX." />

<!-- URL Validation -->
<asp:TextBox ID="txtURL" runat="server" />
<asp:RegularExpressionValidator ID="revURL" runat="server"
    ControlToValidate="txtURL"
    ValidationExpression="^http(s)?://([\\w-]+\\.)+[\\w-]+(/[\\w- ./?%&=]*)?$"
    ErrorMessage="Enter a valid URL." />

```

2.7 CustomValidator

Allows custom validation logic on both client (JavaScript) and server (VB.NET).

```
<asp:TextBox ID="txtCustom" runat="server" />
<asp:CustomValidator ID="cvCustom" runat="server"
    ControlToValidate="txtCustom"
    ClientValidationFunction="validateEven"
    OnServerValidate="cvCustom_ServerValidate"
    ErrorMessage="Value must be an even number." />

<script type="text/javascript">
function validateEven(sender, args) {
    args.IsValid = (parseInt(args.Value) % 2 === 0);
}
</script>
```

```
' VB.NET Server-Side Validation
Protected Sub cvCustom_ServerValidate(ByVal source As Object, ByVal args As
ServerValidateEventArgs)
    Dim val As Integer
    If Integer.TryParse(args.Value, val) Then
        args.IsValid = (val Mod 2 = 0)
    Else
        args.IsValid = False
    End If
End Sub
```

2.8 ValidationSummary

Displays a summary of all validation errors in one place. Properties: ShowSummary , ShowMessageBox , DisplayMode (BulletList, List, SingleParagraph).

```
<asp:ValidationSummary ID="vsSummary" runat="server"
    HeaderText="Please correct the following errors:"
    DisplayMode="BulletList" ShowSummary="true" ShowMessageBox="false"
    ForeColor="Red" />
```

2.9 Validation Controls Summary Table

Validator	Purpose	Key Properties
RequiredFieldValidator	Check if field has value	ControlToValidate, ErrorMessage, InitialValue
CompareValidator	Compare two values	ControlToCompare, ValueToCompare, Operator, Type
RangeValidator	Check if value in range	MinimumValue, MaximumValue, Type
RegularExpressionValidator	Check pattern match	ValidationExpression
CustomValidator	Custom validation logic	ClientValidationFunction, OnServerValidate
ValidationSummary	Show all errors	ShowSummary, ShowMessageBox, DisplayMode

3. Data Binding

3.1 Theory

Data binding connects a data source (database, collection, XML) to server controls. The `DataSource` property specifies the data source, and `DataBind()` method binds the data to the control.

3.2 DataSource and DataBind Method

```
' Binding a DataTable to a GridView
Dim dt As DataTable = GetDataFromDatabase()
gvProducts.DataSource = dt
gvProducts.DataBind()
```

```
' Binding a List to a DropDownList
Dim cities As New List(Of String)()
cities.Add("Delhi")
cities.Add("Mumbai")
cities.Add("Chennai")
ddlCity.DataSource = cities
ddlCity.DataBind()
```

3.3 Data Binding Expressions

Eval — one-way (read-only) binding. Reads data from the data source.

```
<asp:Label ID="lblName" runat="server" Text='<%# Eval("ProductName") %>' />
<asp:Label ID="lblPrice" runat="server" Text='<%# Eval("Price", "{0:C}") %>' />
```

Bind — two-way binding. Data can be read and updated.

```
<asp:TextBox ID="txtName" runat="server" Text='<%# Bind("ProductName") %>' />
<asp:TextBox ID="txtPrice" runat="server" Text='<%# Bind("Price") %>' />
```

3.4 DataBinder.Eval

A static method that evaluates late-bound data expressions. Alternative to `Eval` with optional format string.

```
' Inline usage
<%# DataBinder.Eval(Container.DataItem, "ProductName") %>
<%# DataBinder.Eval(Container.DataItem, "Price", "{0:N2}") %>

' In code-behind
Dim name As String = DataBinder.Eval(e.Row.DataItem,
"ProductName").ToString()
```

3.5 Complete Data Binding Example

```
<asp:GridView ID="gvProducts" runat="server" AutoGenerateColumns="false">
  <Columns>
    <asp:BoundField DataField="ProductID" HeaderText="ID" />
    <asp:TemplateField HeaderText="Name">
      <ItemTemplate>
        <%# Eval("ProductName") %>
      </ItemTemplate>
    </asp:TemplateField>
    <asp:TemplateField HeaderText="Price">
      <ItemTemplate>
        <%# Eval("Price", "{0:C}") %>
      </ItemTemplate>
      <EditItemTemplate>
        <asp:TextBox ID="txtPrice" runat="server" Text='<%#
Bind("Price") %>' />
      </EditItemTemplate>
    </asp:TemplateField>
  </Columns>
</asp:GridView>
```

```
' Code-Behind
Protected Sub Page_Load(ByVal sender As Object, ByVal e As EventArgs) Handles
Me.Load
  If Not IsPostBack Then BindData()
End Sub

Private Sub BindData()
  Dim dt As New DataTable()
```

```
dt.Columns.Add("ProductID", GetType(Integer))  
dt.Columns.Add("ProductName", GetType(String))  
dt.Columns.Add("Price", GetType(Decimal))  
dt.Rows.Add(1, "Laptop", 45000)  
dt.Rows.Add(2, "Mouse", 500)  
dt.Rows.Add(3, "Keyboard", 1200)  
gvProducts.DataSource = dt  
gvProducts.DataBind()  
End Sub
```

4. Data Controls

4.1 GridView

The GridView control displays data in a table format with built-in features: paging, sorting, editing, deleting, and selecting.

4.1.1 GridView Columns

```
<asp:GridView ID="gvEmployees" runat="server" AutoGenerateColumns="False"
    DataKeyNames="EmpID" AllowPaging="True" PageSize="5"
    AllowSorting="True" CssClass="table">
    <Columns>
        <asp:BoundField DataField="EmpID" HeaderText="ID" ReadOnly="True"
SortExpression="EmpID" />
        <asp:BoundField DataField="Name" HeaderText="Employee Name"
SortExpression="Name" />
        <asp:BoundField DataField="Department" HeaderText="Dept"
SortExpression="Department" />
        <asp:BoundField DataField="Salary" HeaderText="Salary"
DataFormatString="{0:N2}" />
        <asp:CommandField ShowEditButton="True" ShowDeleteButton="True" />
    </Columns>
    <HeaderStyle BackColor="#1a5276" ForeColor="White" />
    <AlternatingRowStyle BackColor="#f0f4f8" />
    <PagerSettings Mode="NumericFirstLast" PageButtonCount="5" />
    <PagerStyle BackColor="#eaf2f8" />
</asp:GridView>
```

4.1.2 Paging

```
' VB.NET Page Index Changing
Protected Sub gvEmployees_PageIndexChanging(ByVal sender As Object, ByVal e
As GridViewPageEventArgs)
    gvEmployees.PageIndex = e.NewPageIndex
    BindGridView()
End Sub
```

4.1.3 Sorting

```
' VB.NET Sorting
Protected Sub gvEmployees_Sorting(ByVal sender As Object, ByVal e As
GridViewSortEventArgs)
    Dim dv As DataView = GetEmployeeData().DefaultView
    dv.Sort = e.SortExpression & " " & GetSortDirection(e.SortExpression)
    gvEmployees.DataSource = dv
    gvEmployees.DataBind()
End Sub
```

4.2 DataList

The DataList control displays data using templates and supports selection, editing, and paging. It renders items in a format you define using templates.

4.2.1 DataList Templates

```
<asp:DataList ID="dlProducts" runat="server" RepeatColumns="3"
RepeatDirection="Horizontal" CellPadding="10">
    <HeaderTemplate>
        <h3>Product Catalog</h3>
    </HeaderTemplate>
    <ItemTemplate>
        <div class="product-card">
            <h4><%# Eval("ProductName") %></h4>
            <p>Price: <%# Eval("Price", "{0:C}") %></p>
        </div>
    </ItemTemplate>
    <AlternatingItemTemplate>
        <div class="product-card-alt">
            <h4><%# Eval("ProductName") %></h4>
            <p>Price: <%# Eval("Price", "{0:C}") %></p>
        </div>
    </AlternatingItemTemplate>
    <SeparatorTemplate>
        <hr />
    </SeparatorTemplate>
    <FooterTemplate>
        <p>End of catalog</p>
    </FooterTemplate>
</asp:DataList>
```



```
' VB.NET
Protected Sub Page_Load(ByVal sender As Object, ByVal e As EventArgs) Handles
Me.Load
    If Not IsPostBack Then BindDataList()
End Sub

Private Sub BindDataList()
    dlProducts.DataSource = GetProducts()
    dlProducts.DataBind()
End Sub
```

4.3 Repeater

The Repeater control displays repeated data using templates with complete control over the HTML output. It does not add any extra HTML like GridView or DataList.

4.3.1 Repeater Templates

```
<asp:Repeater ID="rptEmployees" runat="server"
OnItemDataBound="rptEmployees_ItemDataBound">
    <HeaderTemplate>
        <table border="1">
            <tr><th>ID</th><th>Name</th><th>City</th></tr>
        </HeaderTemplate>
        <ItemTemplate>
            <tr>
                <td><%# Eval("EmpID") %></td>
                <td><%# Eval("Name") %></td>
                <td><%# Eval("City") %></td>
            </tr>
        </ItemTemplate>
        <AlternatingItemTemplate>
            <tr style="background:#f0f4f8;">
                <td><%# Eval("EmpID") %></td>
                <td><%# Eval("Name") %></td>
                <td><%# Eval("City") %></td>
            </tr>
        </AlternatingItemTemplate>
        <FooterTemplate>
            </table>
        </FooterTemplate>
    </asp:Repeater>
```

```
' VB.NET ItemDataBound Event
Protected Sub rptEmployees_ItemDataBound(ByVal sender As Object, ByVal e As
RepeaterItemEventArgs)
    If e.Item.ItemType = ListItemType.Item OrElse e.Item.ItemType =
ListItemType.AlternatingItem Then
        Dim drv As DataRowView = DirectCast(e.Item.DataItem, DataRowView)
        ' Custom logic per item
    End If
End Sub
```

4.4 DetailsView

Displays a single record at a time in a table format. Supports insert, update, delete, and paging through records.

```
<asp:DetailsView ID="dvEmployee" runat="server" Height="50px" Width="350px"
AutoGenerateRows="False" DataKeyNames="EmpID"
AllowPaging="True"
HeaderText="Employee Details">
    <Fields>
        <asp:BoundField DataField="EmpID" HeaderText="ID" ReadOnly="True" />
        <asp:BoundField DataField="Name" HeaderText="Name" />
        <asp:BoundField DataField="Department" HeaderText="Department" />
        <asp:BoundField DataField="Salary" HeaderText="Salary"
DataFormatString="{0:N2}" />
        <asp:CommandField ShowEditButton="True" ShowDeleteButton="True"
ShowInsertButton="True" />
    </Fields>
</asp:DetailsView>
```

4.5 FormView

Similar to DetailsView but uses templates for complete control over the display of each record. Supports insert, update, and delete.

```
<asp:FormView ID="fvProduct" runat="server" DataKeyNames="ProductID"
AllowPaging="True">
    <ItemTemplate>
        <div class="product-detail">
            <h3><%# Eval("ProductName") %></h3>
            <p>Price: <%# Eval("Price", "{0:C}") %></p>
            <p>Category: <%# Eval("Category") %></p>
```

```
        <asp:LinkButton ID="btnEdit" runat="server" CommandName="Edit"
Text="Edit" />
        <asp:LinkButton ID="btnDelete" runat="server"
CommandName="Delete" Text="Delete" />
    </div>
</ItemTemplate>
<EditItemTemplate>
    <div class="edit-form">
        Name: <asp:TextBox ID="txtName" runat="server" Text='<%#
Bind("ProductName") %>' />
        Price: <asp:TextBox ID="txtPrice" runat="server" Text='<%#
Bind("Price") %>' />
        <asp:LinkButton ID="btnUpdate" runat="server"
CommandName="Update" Text="Update" />
        <asp:LinkButton ID="btnCancel" runat="server"
CommandName="Cancel" Text="Cancel" />
    </div>
</EditItemTemplate>
</asp:FormView>
```

5. Master Pages

5.1 Theory

Master Pages provide a template for consistent layout across all pages of a web application. A master page has a `.master` extension and contains **ContentPlaceHolder** controls where content pages inject their content.

5.2 Master Page (.master file)

```
<%@ Master Language="VB" AutoEventWireup="false" CodeFile="Site.master.vb"
Inherits="SiteMaster" %>
<!DOCTYPE html>
<html>
<head runat="server">
    <title><asp:ContentPlaceHolder ID="head" runat="server">Default
Title</asp:ContentPlaceHolder></title>
    <link rel="stylesheet" href="styles.css" />
</head>
<body>
    <form id="form1" runat="server">
        <div id="header">
            <h1>My ASP.NET Application</h1>
        </div>
        <div id="nav">
            <ul>
                <li><a href="Default.aspx">Home</a></li>
                <li><a href="About.aspx">About</a></li>
                <li><a href="Contact.aspx">Contact</a></li>
            </ul>
        </div>
        <div id="main">
            <asp:ContentPlaceHolder ID="MainContent" runat="server">
                <!-- Default content here -->
            </asp:ContentPlaceHolder>
        </div>
        <div id="footer">
            &copy; 2024 My ASP.NET Application
        </div>
    </form>
```

```
</body>  
</html>
```

5.3 Content Page (with MasterPageFile attribute)

```
<%@ Page Title="Home" Language="VB" MasterPageFile="~/Site.master"  
AutoEventWireup="false"  
CodeFile="Default.aspx.vb" Inherits="_Default" %>  
  
<asp:Content ID="Content1" ContentPlaceHolderID="head" runat="server">  
    <!-- Page-specific head content -->  
</asp:Content>  
  
<asp:Content ID="Content2" ContentPlaceHolderID="MainContent" runat="server">  
    <h2>Welcome to the Home Page</h2>  
    <p>This content is placed in the MainContent placeholder.</p>  
</asp:Content>
```

5.4 Nested Master Pages

A master page can itself be a content page of another master page, creating a hierarchy.

```
<!-- Parent Master: Site.master -->  
<%@ Master Language="VB" AutoEventWireup="false" CodeFile="Site.master.vb"  
Inherits="SiteMaster" %>  
<html>  
<body>  
    <form id="form1" runat="server">  
        <div id="header">Header</div>  
        <asp:ContentPlaceHolder ID="MainContent" runat="server" />  
        <div id="footer">Footer</div>  
    </form>  
</body>  
</html>  
  
<!-- Child Master: Products.master -->  
<%@ Master Language="VB" MasterPageFile="~/Site.master"  
AutoEventWireup="false"  
CodeFile="Products.master.vb" Inherits="Products" %>  
  
<asp:Content ID="Content1" ContentPlaceHolderID="MainContent" runat="server">  
    <div id="sidebar">
```

```
        <ul><li>Category 1</li><li>Category 2</li></ul>
    </div>
    <div id="content-area">
        <asp:ContentPlaceHolder ID="ProductContent" runat="server" />
    </div>
</asp:Content>

<!-- Content Page using Child Master -->
<%@ Page Title="Laptops" Language="VB" MasterPageFile="~/Products.master" %>
<asp:Content ID="Content2" ContentPlaceHolderID="ProductContent"
runat="server">
    <h2>Laptops</h2>
    <!-- Product listing -->
</asp:Content>
```

6. Navigation

6.1 Theory

ASP.NET provides three navigation controls: **SiteMapPath** (breadcrumb), **Menu**, and **TreeView**. They all use a `web.sitemap` XML file as their data source.

6.2 web.sitemap File

```
<?xml version="1.0" encoding="utf-8" ?>
<siteMap xmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0">
  <siteMapNode url="~/Default.aspx" title="Home" description="Home">
    <siteMapNode url="~/Products.aspx" title="Products" description="Our
Products">
      <siteMapNode url="~/Laptops.aspx" title="Laptops"
description="Laptops" />
      <siteMapNode url="~/Mobiles.aspx" title="Mobiles"
description="Mobiles" />
    </siteMapNode>
    <siteMapNode url="~/About.aspx" title="About Us" description="About
Us">
      <siteMapNode url="~/Team.aspx" title="Our Team" />
      <siteMapNode url="~/History.aspx" title="History" />
    </siteMapNode>
    <siteMapNode url="~/Contact.aspx" title="Contact"
description="Contact Us" />
  </siteMapNode>
</siteMap>
```

6.3 SiteMapPath (Breadcrumb)

Displays a breadcrumb trail showing the current page's position in the site hierarchy.

```
<asp:SiteMapPath ID="smpBreadcrumb" runat="server"
  PathSeparator=" &gt; "
  Font-Names="Segoe UI"
  PathDirection="RootToCurrent"
  RenderCurrentNodeAsLink="false"
```

```
NodeStyle-Font-Bold="true"  
CurrentNodeStyle-Color="#1a5276"  
RootNodeStyle-Color="#2e86c1" />
```

6.4 Menu Control

Displays a hierarchical menu. Supports `Orientation` (Horizontal/Vertical), `StaticDisplayLevels`, and `MaximumDynamicDisplayLevels`.

```
<asp:Menu ID="menuNav" runat="server" DataSourceID="smdsNav"  
    Orientation="Horizontal"  
    StaticDisplayLevels="2"  
    MaximumDynamicDisplayLevels="3"  
    StaticMenuStyle-BackColor="#1a5276"  
    StaticMenuItemStyle-ForeColor="White"  
    DynamicMenuStyle-BackColor="#eaf2f8"  
    DynamicMenuItemStyle-ForeColor="#1a5276">  
</asp:Menu>  
  
<asp:SiteMapDataSource ID="smdsNav" runat="server" ShowStartingNode="true" />
```

6.5 TreeView Control

Displays hierarchical data in a tree structure. Supports expand/collapse, checkboxes, and custom icons.

```
<asp:TreeView ID="tvNav" runat="server" DataSourceID="smdsTree"  
    ShowLines="True" ExpandDepth="1"  
    NodeStyle-Font-Names="Segoe UI"  
    NodeStyle-Font-Size="11pt"  
    NodeIndent="20">  
    <NodeStyle CssClass="tree-node" />  
    <ParentNodeStyle Font-Bold="True" ForeColor="#1a5276" />  
    <LeafNodeStyle ForeColor="#2e86c1" />  
    <HoverNodeStyle BackColor="#eaf2f8" />  
</asp:TreeView>  
  
<asp:SiteMapDataSource ID="smdsTree" runat="server" />
```

6.6 Navigation Controls Summary

Control	Display	Use Case
SiteMapPath	Breadcrumb trail	Show current page location
Menu	Hierarchical menu (static + dynamic)	Main navigation bar
TreeView	Expandable tree structure	Side navigation, file explorer

7. Themes & Skins

7.1 Theory

Themes provide a unified look and feel across an entire web application. They consist of **.skin** files (control property definitions), CSS files, and images stored inside the `App_Themes` folder.

7.2 .skin File Structure

```
<!-- App_Themes/BlueTheme/Blue.skin -->
<!-- Default skin for all controls of a type -->
<asp:TextBox runat="server" BackColor="#f0f4f8" BorderColor="#2e86c1"
BorderWidth="1px" />
<asp:Button runat="server" BackColor="#1a5276" ForeColor="White" Font-
Bold="True"
    BorderStyle="None" Width="100px" />
<asp:Label runat="server" ForeColor="#1a5276" Font-Names="Segoe UI" />
<asp:GridView runat="server" BackColor="White" BorderColor="#2e86c1"
    HeaderStyle-BackColor="#1a5276" HeaderStyle-ForeColor="White"
    AlternatingRowStyle-BackColor="#f0f4f8" />

<!-- Named skin (applied via SkinID) -->
<asp:Button runat="server" SkinID="DangerButton" BackColor="Red"
ForeColor="White" />
<asp:Button runat="server" SkinID="SuccessButton" BackColor="Green"
ForeColor="White" />
```

7.3 Applying a Theme

In **web.config** (applies to all pages):

```
<configuration>
  <system.web>
    <pages theme="BlueTheme" />
  </system.web>
</configuration>
```

In **page directive** (applies to one page):

```
<%@ Page Theme="BlueTheme" %>
```

7.4 Theme vs StyleSheetTheme

Feature	Theme	StyleSheetTheme
Override priority	Overrides control properties (highest)	Can be overridden by control properties
Page directive	Theme="ThemeName"	StyleSheetTheme="ThemeName"
Control-level override	Cannot override Theme values on controls	Control properties override StyleSheetTheme
Use case	Enforce a consistent look	Default styling that can be customized

```
<!-- StyleSheetTheme allows individual control overrides -->
<%@ Page StyleSheetTheme="BlueTheme" %>

<!-- This BackColor will override the skin -->
<asp:TextBox ID="txtSpecial" runat="server" BackColor="Yellow" />
```

7.5 Using SkinID

```
<asp:Button ID="btnDelete" runat="server" SkinID="DangerButton" Text="Delete" />
<asp:Button ID="btnSave" runat="server" SkinID="SuccessButton" Text="Save" />
```

8. Globalization & Localization

8.1 Theory

- **Globalization** — designing applications that support multiple cultures and languages.
- **Localization** — adapting a globalized application to a specific culture/locale.

8.2 Setting Culture and UICulture

In **web.config**:

```
<configuration>
  <system.web>
    <globalization culture="en-IN" uiCulture="en-IN" />
  </system.web>
</configuration>
```

In **page directive**:

```
<%@ Page Culture="auto" UICulture="auto" %>
```

In **code-behind**:

```
Protected Overrides Sub InitializeCulture()
    Dim lang As String = Request.QueryString("lang")
    If Not String.IsNullOrEmpty(lang) Then
        System.Threading.Thread.CurrentThread.CurrentCulture = _
            New System.Globalization.CultureInfo(lang)
        System.Threading.Thread.CurrentThread.CurrentUICulture = _
            New System.Globalization.CultureInfo(lang)
    End If
End Sub
```

8.3 Resource Files (.resx)

Resource files store locale-specific strings, images, and other data. Common files:

- App_GlobalResources/Resources.resx (default/English)
- App_GlobalResources/Resources.hi-IN.resx (Hindi)
- App_GlobalResources/Resources.fr.resx (French)

```
' Using resources in code-behind
lblWelcome.Text = Resources.Resource.WelcomeMessage
btnSubmit.Text = Resources.Resource.SubmitButton
```

```
<!-- Using resources in markup -->
<asp:Label ID="lblWelcome" runat="server"
    Text='<%$ Resources:Resource, WelcomeMessage %>' />
<asp:Button ID="btnSubmit" runat="server"
    Text='<%$ Resources:Resource, SubmitButton %>' />
```

8.4 Localizing Dates and Numbers

```
' Format dates per culture
Dim dt As DateTime = DateTime.Now
lblDate.Text = dt.ToString("D") ' Long date format for current culture

' Format currency per culture
Dim price As Decimal = 1234.56D
lblPrice.Text = price.ToString("C") ' Currency symbol per culture
```

9. Security

9.1 Theory

ASP.NET provides multiple authentication and authorization mechanisms: Windows Authentication, Forms Authentication, Membership, Roles, and Login Controls.

9.2 Authentication Modes

Mode	Description	Use Case
Windows	Uses Windows user accounts (Active Directory)	Intranet applications
Forms	Uses a login form and authentication ticket (cookie)	Internet/web applications
Passport	Uses Microsoft Passport Service	Legacy/delegated auth
None	No authentication	Public/anonymous sites

9.3 Windows Authentication

Configured in **web.config**:

```
<configuration>
  <system.web>
    <authentication mode="Windows" />
    <authorization>
      <deny users="?" /> <!-- Deny anonymous users -->
    </authorization>
  </system.web>
</configuration>
```

```
' Retrieving Windows identity
Dim winUser As System.Security.Principal.WindowsIdentity = _
```

```
System.Security.Principal.WindowsIdentity.GetCurrent()  
lblUser.Text = "User: " & User.Identity.Name
```

9.4 Forms Authentication

Configured in **web.config**:

```
<configuration>  
  <connectionStrings>  
    <add name="MembershipConn" connectionString="..." />  
  </connectionStrings>  
  <system.web>  
    <authentication mode="Forms">  
      <forms loginUrl="Login.aspx" timeout="30"  
        defaultUrl="Default.aspx" slidingExpiration="true" />  
    </authentication>  
    <authorization>  
      <deny users="?" /> <!-- Anonymous users redirected to login -->  
    </authorization>  
  </system.web>  
</configuration>
```

9.5 Membership

ASP.NET Membership manages user accounts (create, validate, manage passwords).

```
' Create a new user  
Dim status As MembershipCreateStatus  
Dim user As MembershipUser = Membership.CreateUser("john", "P@ss123", _  
  "john@example.com", "Question?", "Answer", True, status)  
  
' Validate user credentials  
If Membership.ValidateUser("john", "P@ss123") Then  
  FormsAuthentication.RedirectFromLoginPage("john", False)  
End If  
  
' Get user information  
Dim currentUser As MembershipUser = Membership.GetUser(User.Identity.Name)  
lblEmail.Text = currentUser.Email  
  
' Configure membership in web.config  
<membership defaultProvider="SqlMembershipProvider">  
  <providers>
```

```

        <add name="SqlMembershipProvider"
            type="System.Web.Security.SqlMembershipProvider"
            connectionStringName="MembershipConn"
            enablePasswordRetrieval="false"
            enablePasswordReset="true"
            requiresQuestionAndAnswer="true"
            minRequiredPasswordLength="6" />
    </providers>
</membership>

```

9.6 Roles

Role-based authorization manages groups of users.

```

' Create roles and assign users
If Not Roles.RoleExists("Admin") Then Roles.CreateRole("Admin")
If Not Roles.RoleExists("Users") Then Roles.CreateRole("Users")
Roles.AddUserToRole("john", "Admin")
Roles.AddUserToRole("jane", "Users")

' Check role membership
If User.IsInRole("Admin") Then
    ' Show admin-only content
End If

' Configure roles in web.config
<roleManager enabled="true" defaultProvider="SqlRoleProvider">
    <providers>
        <add name="SqlRoleProvider"
            type="System.Web.Security.SqlRoleProvider"
            connectionStringName="MembershipConn" />
    </providers>
</roleManager>

' Restrict access by role
<authorization>
    <allow roles="Admin" />
    <deny users="*" />
</authorization>

```

9.7 Login Controls

ASP.NET provides ready-made login controls that work with the Membership system.


```
<!-- Login Control -->
<asp:Login ID="Login1" runat="server"
    TitleText="Please Log In"
    DestinationPageUrl="~/Default.aspx"
    FailureText="Invalid username or password."
    LoginButtonText="Sign In"
    RememberMeText="Keep me logged in" />

<!-- LoginView Control (shows different content for authenticated/anonymous
users) -->
<asp:LoginView ID="LoginView1" runat="server">
    <AnonymousTemplate>
        Welcome, guest! <asp:LoginStatus ID="LoginStatus1" runat="server" />
    </AnonymousTemplate>
    <LoggedInTemplate>
        Welcome, <asp:LoginName ID="LoginName1" runat="server" />!
        <asp:LoginStatus ID="LoginStatus2" runat="server" LogoutText="Logout"
/>
    </LoggedInTemplate>
    <RoleGroups>
        <asp:RoleGroup Roles="Admin">
            <ContentTemplate>
                Welcome Admin! <a href="Admin/">Admin Panel</a>
            </ContentTemplate>
        </asp:RoleGroup>
    </RoleGroups>
</asp:LoginView>

<!-- CreateUserWizard Control -->
<asp:CreateUserWizard ID="CreateUserWizard1" runat="server"
    ContinueDestinationPageUrl="~/Default.aspx">
    <WizardSteps>
        <asp:CreateUserWizardStep ID="Step1" runat="server" Title="Account
Info">
            <ContentTemplate>
                Username: <asp:TextBox ID="UserName" runat="server" />
                Password: <asp:TextBox ID="Password" runat="server"
TextMode="Password" />
                Email: <asp:TextBox ID="Email" runat="server" />
            </ContentTemplate>
        </asp:CreateUserWizardStep>
        <asp:CompleteWizardStep ID="Step2" runat="server" Title="Complete" />
    </WizardSteps>
</asp:CreateUserWizard>

<!-- PasswordRecovery Control -->
<asp:PasswordRecovery ID="PasswordRecovery1" runat="server" />
```

```
<!-- ChangePassword Control -->  
<asp:ChangePassword ID="ChangePassword1" runat="server" />
```

9.8 Security Best Practices

- Use strong password policies (min length, complexity)
- SSL for login pages
- Use parameterized queries to prevent SQL injection
- Validate all user input
- Use AntiForgeryToken to prevent CSRF
- Implement proper error handling (custom error pages)
- Log security events

10. Questions

Section A: List Controls

1. Explain the difference between DropDownList and ListBox with examples.
2. How do you implement multiple selection in a ListBox?
3. Write the ASP.NET code to create a CheckBoxList and retrieve selected items in VB.NET code-behind.
4. What is the purpose of the RadioButtonList control? How is it different from CheckBoxList?
5. Explain the BulletedList control with its DisplayMode property.
6. Compare the five simple list controls in ASP.NET with their selection modes.

Section B: Validation Controls

1. Explain the common properties of all validation controls (ControlToValidate, ErrorMessage, Display, Text, SetFocusOnError).
2. Write a complete validation example using RequiredFieldValidator, RangeValidator, CompareValidator, and RegularExpressionValidator.
3. What is the difference between Display="Static" and Display="Dynamic" in validation controls?
4. Explain CustomValidator with a client-side JavaScript function and server-side VB.NET handler.
5. How does ValidationSummary work? Explain its ShowSummary and ShowMessageBox properties.
6. Write a registration form with complete validation for Name, Email, Age, Password, and Confirm Password.

Section C: Data Binding & Data Controls

1. Explain the DataBind method and how it works with different data sources.
2. What is the difference between Eval and Bind in data binding expressions?
3. Explain DataBinder.Eval with a code example.
4. Describe the GridView control. How do you implement paging and sorting?
5. What are the different templates available in DataList and Repeater controls? Explain with examples.
6. Compare DetailsView and FormView controls with code examples.
7. Write a complete example showing how to bind a DataTable to a GridView.

Section D: Master Pages, Navigation, Themes & Security

1. What is a Master Page in ASP.NET? Explain its purpose with a complete example.
2. How do you create and use nested master pages?
3. Explain the ContentPlaceHolder control and the MasterPageFile attribute.
4. Describe the web.sitemap file structure. How do SiteMapPath, Menu, and TreeView use it?
5. What are Themes and Skins? Explain the difference between Theme and StyleSheetTheme.
6. How are .skin files created and applied? Explain the SkinID attribute.
7. Explain the difference between Windows and Forms authentication in ASP.NET.
8. Describe ASP.NET Membership and Role management with code examples.
9. List and explain the built-in Login controls in ASP.NET.
10. Explain how Globalization and Localization work in ASP.NET (culture, resource files, .resx).

Section E: Long Questions (10 marks)

1. Design a complete Employee Registration Form using all list controls, validation controls, and a GridView to display submitted data. Provide complete ASP.NET and VB.NET code-behind.
2. Explain the architecture of Master Pages in ASP.NET. Create a master page with header, navigation (using Menu control), content area, and footer. Create two content pages that use this master page.
3. Write a detailed note on security in ASP.NET. Explain Forms authentication, Membership, Roles, and the Login controls with complete configuration and code examples.
4. Compare all data controls in ASP.NET: GridView, DataList, Repeater, DetailsView, and FormView. Include features like templates, paging, sorting, editing, and use cases for each.

ASP.NET Unit 4: Advanced Web Forms — Comprehensive Study Notes for HPU BCA Semester V